



MULTIMEDIA-PROGRAMAZIOA ETA GAILU MUGIKORRAK

MVC (MODELO-VISTA-CONTROLADOR)

1. MVC KONTZEPTUA

- **View (Bista):** erabiltzaileak ikusten duen interfazea da (Activity + XML).
- **Controller (Kontrolatzailea):** bi munduen arteko zubia da; erabiltzailearen ekintzak interpretatzen ditu eta modeloa eguneratzen du.
- **Model (Modeloa):** datuak eta logika gordetzen ditu (zenbaketa, kalkuluak, etab.).

Horrela **erantzukizunak banatzen dira**, eta kodea garbi eta mantengarriagoa da.

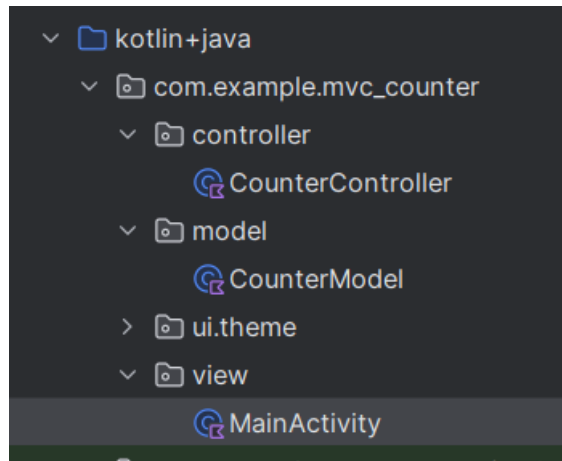
2. PROIEKTUA SORTZEA ANDROID STUDIO-N BISTA ETA KONTROLATZAILEA BANANDUTA

1. Ireki **Android Studio** → **New Project** → *Empty Activity*.
2. Izena: **MVC_Counter**.
3. Hizkuntza: **Kotlin**.
4. Minimum **SDK: API 21** edo handiagoa.
5. Sakatu **Finish** eta itxaron Gradle sinkronizatu arte.

A. PAKETE-EGITURA GOMENDATUA

app\src\main\java\com\example\mvc_counter azpian sortu hiru azpifitxategi:

```
com.example.mvc_counter
├── model
│   └── CounterModel.kt
├── controller
│   └── CounterController.kt
└── view
    └── MainActivity.kt + activity_main.xml
```



Hau da MVC banaketa klasikoa:

- **Model:** CounterModel.kt
- **Controller:** CounterController.kt
- **View:** MainActivity.kt eta activity_main.xml

B. ADIBIDEA: ZENBATZAILE SINPLEA (ACTIVITY_MAIN.XML)



C. ADIBIDEA: ZENBATZAILE SINPLEA (MAINACTIVITY.KT)

❖ ALDAGAI PRIBATUAK: LATEINIT ERABILERA

```
2 Usages
private lateinit var tvCount: TextView
2 Usages
private lateinit var btnInc: Button
2 Usages
private lateinit var btnDec: Button
2 Usages
private lateinit var btnReset: Button

2 Usages
private lateinit var model: CounterModel
4 Usages
private lateinit var controller: CounterController
```

Lateinit-en bitartez aldagaia definitzen da baina ez da hasieratzen. Normalean **findViewById**-rekin edo **new**-n bitartez hasieratzen dira **onCreate**-n barnean.

Kontuz: **lateinit** aldagai bat null balioa badu *UninitializedPropertyAccessException* errorea ager daiteke, beraz hasieratzea ezinbestekoa da.

❖ ONCREATE

Activity-ren hasiera eta UI loturak.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
}
```

onCreate Activity-ren sarrera-puntua da: hemen jarri behar dira layout-a eta hasieraketa gehiena.

setContentView(...) bidez **activity_main.xml** kargatzen da; ondoren **findViewById** erabil dezakegu xml-ko elementuak harrapatzeko.

❖ VIEW ELEMENTUAK ESKURATZEA

```
// VIEW elementuak hartu  
tvCount = findViewById( id = R.id.tvCount)  
btnInc = findViewById( id = R.id.btnInc)  
btnDec = findViewById( id = R.id.btnDec)  
btnReset = findViewById( id = R.id.btnReset)
```

Hemen XML-n definitutako elementu guztiak (TextView eta botoiak) Kotlin-eko objektu bihurtzen dira. **Garrantzitsua:** **findViewById** egin behin **setContentView** egin eta gero, bestela ez du elementurik aurkituko eta erroreak agertuko dira.

❖ CONTROLLER SORTU ETA HASIERAKO BALIOA BISTARATU

```
controller = CounterController() // Controller sortzen du
//Hasierako balioa ikusteko
controller.attachView( v = this)
```

❖ BOTOIEN EKINTZAK: CONTROLLER BIDEZ KONTROLATZEA

```
// CONTROLLER botoiaren ekintzak
btnInc.setOnClickListener { controller.onIncrementClicked() }
btnDec.setOnClickListener { controller.onDecrementClicked() }
btnReset.setOnClickListener { controller.onResetClicked() }
```

Edozein botoi sakatzean ez duzu zuzenean tvCount aldatzen, **Controller**-i esaten diozu zer egin (intentzioa). **Controller**-ek ondoren **Model**-ari deitzen dio (model.increment() adibidez). Hemendik abiatzen da **MVC**-ren oinarria:

- **View** = input eta rendering;
- **Controller** = ekintza prozesatu;
- **Model** = datua eguneratu.

❖ CONTROLLER : VIEW KOMUNIKAZIOA

```
override fun showCount(count: Int) {
    tvCount.text = count.toString()
}
```

Hori da Activity-k aplikatzen duen CounterView interfazearen metodoa.

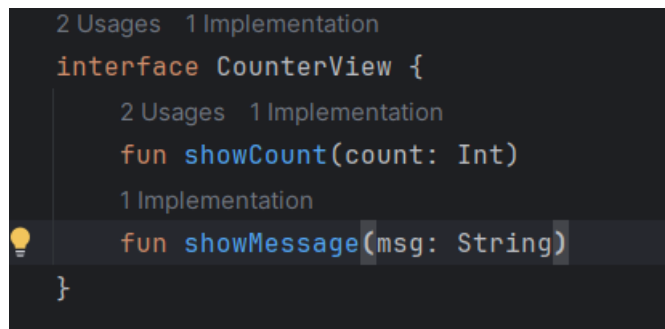
Kontrolatzaileak view.showCount(newCount) deitzen du Model-etik etorritako aldaketaren ondoren.

Ondoren, Activity-k TextView eguneratzen du UI-n. Gogoratu: UI eguneraketak beti burutu behar dira main thread-ean — hemen ondo dago (Activity metodoetan gaude).

D. CONTROLLER

Controller-ak bistatik (botoiak, testuak...) jasotzen ditu ekintzak, eta **model**-ari esaten dio zer egin behar duen (gehitu, kendu, reset...). Ondoren, modeloak kontrolatzaileari bueltan esaten dio: *“balioa aldatu dut”*, eta controller-ak bistari bidaltzen dio informazio hori eguneratzeko.

❖ BARNE INTERFAZEA (COUNTERVIEW)



```
2 Usages 1 Implementation
interface CounterView {
    2 Usages 1 Implementation
    fun showCount(count: Int)
    1 Implementation
    fun showMessage(msg: String)
}
```

Hau da **view**-ekin egiten den kontratu txiki bat da. Controller-ak ez du Activity konkretua ezagutzen, baina badaki bistak bi metodo hauek implementatu behar dituela:

- **showCount(count: Int)**: zenbaki berria erakusteko.
- **showMessage(msg: String)**: mezu bat (Toast edo Snackbar) erakusteko.

Hau onuragarria da Activity aldatzen baduzu (beste diseinu bateko bat jarritz, adibidez tablet baterako), ez duzu controller aldatu behar, kontratua berdina delako.

❖ INIT BLOKEAN: HASIERAKO KONFIGURAZIOA

```
5 Usages
private val model = CounterModel() // Kontroladoreak sortzen du m
4 Usages
private var view: CounterView? = null

init {
    model.addListener( | = object : CounterModel.Listener {
        override fun onCountChanged(newCount: Int) {
            view?.showCount(newCount)
        }
    })
}
```

init blokea exekutatzen da **controller-a sortzen denean**.

model.addListener(): Honek **controller-a** gehitzen du **model-en** “entzule” zerrendara.

CounterModel.Listener Interfaze hau behar da modeloak nori bidali behar dizkion aldaketak definitzeko.

```
override fun onCountChanged(newCount: Int) {
    view?.showCount(newCount)
}
```

Hau da **listener** metodoa. **Model-ak** **notifyListeners()** deitzen duenean, **controller-ak** hau jasotzen du.

view.showCount(newCount) deitzean balioaren aldaketa jaso du, horrek **Activity-n** (bistan) **TextView** eguneratzen du, zenbaki berria erakutsiz.

Kontroladorearen view (MainActivity) **nullable** da, beraz ?. erabiltzen da.

var name: String = "Jon" // Ez da nullable, beti String bat izango da

var surname: String? = null // Nullable da, balio bat edo null izan dezake

❖ METODO NAGUSIAK: BOTOIEN EKINTZAK

```
1 Usage  
fun onIncrementClicked() = model.increment()  
1 Usage  
fun onDecrementClicked() = model.decrement()  
1 Usage  
fun onResetClicked() = model.reset()
```

Garrantzitsua: **Controller**-ak ez du **TextView** zuzenean eguneratzen, ezta **Toast** bat erakusten ere, hori **view**-aren betebeharra da da.

E. MODEL

Model-a da aplikazioaren logika gordetzen duena. Zenbatzailearen balioa aldatzean, “entzungailuei” (listeners) abisatzen die, **view**-ak eguneratu dezaten.

```
class CounterModel {  
    5 Usages  
    private var count: Int = 0
```

- **CounterModel** izeneko klase bat da.
- **count** aldagaia da **zenbaketa balioa** (hasieran 0)gordeko du.
- **private** denez kanpotik ez da zuzenean aldagai hori manipulatu ahal izango, encapsulation-a (datuen babesa).

```
3 Usages 1 Implementation  
interface Listener {  
    1 Usage 1 Implementation  
    fun onCountChanged(newCount: Int)  
}
```

Hemen **interfazea** sortzen dugu, hau da: “*Norbaitek CounterModel-ari entzun nahi badio, metodo hau implementatu behar du*”.

Interfazea: «entzuleei aldaketak nola jakinaraziko zaien definitzen du. Edukiaren aldaketan **onCountChanged** deituko da

```
4 Usages  
private val listeners = mutableListOf<Listener>()
```

Hemen gordetzen dira listener guztiak (objektuak, Activity edo Fragment bat izan daitezke).

- **mutableListOf:** zerrenda aldagarria da, elementuak gehitu edo kendu daitezke.

```
fun addListener(l: Listener) {  
    if (!listeners.contains(l)) listeners.add(l)  
}  
  
fun removeListener(l: Listener) {  
    listeners.remove(element = l)  
}
```

- **addListener():** entzungailu berri bat gehitzen du, baina **bikoiztuta ez badago**.
- **removeListener():** entzungailua kentzen du. Orokorrean, **Activity** edo **controller**-ak **onDestroy/onStop**-en deiatu behar dute hau memoria-ihesa saihesteko.

```
fun getCount(): Int = count
```

count balioa irakurtzeko publiko dagoen metodo laburra (encapsulation mantentzen du).

```
fun increment() {  
    count++  
    notifyListeners()  
}  
  
fun decrement() {  
    count--  
    notifyListeners()  
}  
  
fun reset() {  
    count = 0  
    notifyListeners()  
}
```

- **increment()**: zenbakitzailea 1 gehitzen du.
- **decrement()**: zenbakitzailea 1 kentzen du.
- **reset()**: berriz 0 jartzen du.

Guztiek **notifyListeners()** deitzen dute, aldaketa gertatu dela jakinarazteko.

Horrela, **View** edo **Controller**-ak automatikoki eguneratuko dira balioa aldatzean.

```
3 Usages  
private fun notifyListeners() {  
    for (l in listeners) l.onCountChanged( newCount = count)  
}
```

listeners zerrendako bakoitzari **onCountChanged(count)** deitzen dio. Horrela, **View** edo **Controller**-ak jasotzen dute mezu hau: “*Zenbakitzailea aldatuta dago → eguneratu pantaila*”.

Azalpen orokorra:

- Hemen gordetzen da zenbatzailearen balioa (**count**).
- Metodoak eskaintzen ditu: **increment()**, **decrement()**, **reset()**.

- Balioa aldatzean, “**listener**” edo entzuleei abisatzeko **notifyListeners()** bidez.

F. FUNTZIONAMENDU-ZIKLOA LABURBILDUTA

Adibide baten bitartez **MVC**-baten funtzionamendua azalduko dugu.(Gehiketa)

1. **View** (**MainActivity**)-n **+** **botoia** sakatzean **controller.onIncrementClicked()** funtzioari deia bideratzen da.
2. Momentu hortan **CounterController**-ek **Model**-eko **model.increment()** deitzen du.(**Controller:Model**).
3. **CounterModel**-ek **Modeloa** aldatzen du **count++** eta **notifyListeners()** eginez.
4. Momentu hortan **Controller-ak** jakinarazpena jasotzen du **onCountChanged()** funtzioaren **view.showCount(newCount)** deiaren bidez.
5. Azkenik **MainActivity-ko** **View** eguneratzen da **tvCount.text = count.toString()**-ren bidez